

Text embeddings

Video Link: [Text Embeddings](#)

After breaking a document into chunks, the next step in a RAG pipeline is finding which chunks are most relevant to a user's question. This is essentially a search problem - you need to look through all your text chunks and identify the ones that relate to what the user is asking about.

Finding Relevant Chunks

- This is a search problem!
- How do we find chunks most related to the user's question?

ANTHROPIC

User's Question	How many bugs did engineers fix this year?
------------------------	---

Chunks of Text

This year our company engaged in many areas of research.

Section 1: Medical Research
This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

Section 2: Software Engineering
This division dedicated significant effort to studying various infection vectors in our distributed systems

Semantic Search

The most common approach for finding relevant chunks is semantic search. Unlike keyword-based search that looks for exact word matches, semantic search uses text embeddings to understand the meaning and context of both the user's question and each text chunk.

Semantic Search

Utilize text embeddings to better understand the user's question *and* what each chunk of text is really talking about.

ANTHROPIC

User's
Question

How many bugs did engineers fix
this year?

Chunks of Text

This year our company engaged in many areas of research.

Section 1: Medical Research

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

Section 2: Software Engineering

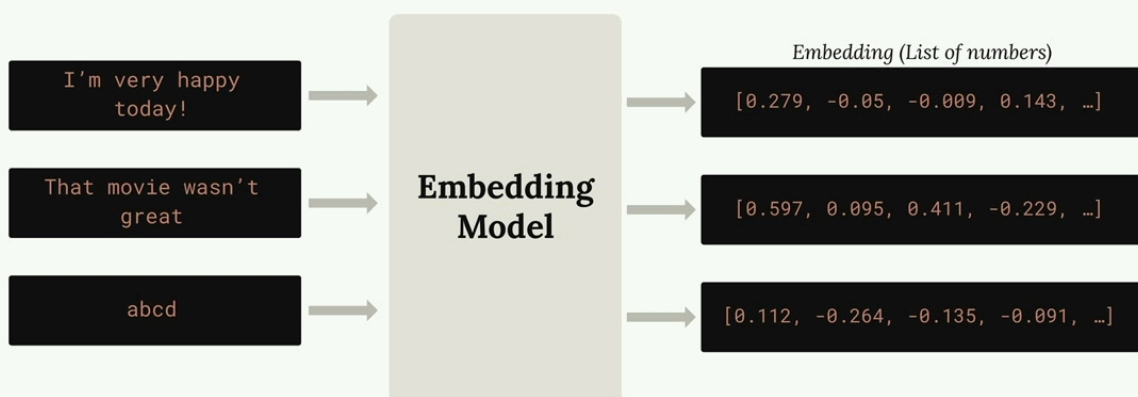
This division dedicated significant effort to studying various infection vectors in our distributed systems

Text Embeddings

A text embedding is a numerical representation of the meaning contained in some text. Think of it as converting words and sentences into a format that computers can work with mathematically.

Text Embeddings

A numerical representation of the meaning contained in some text.



ANTHROPIC

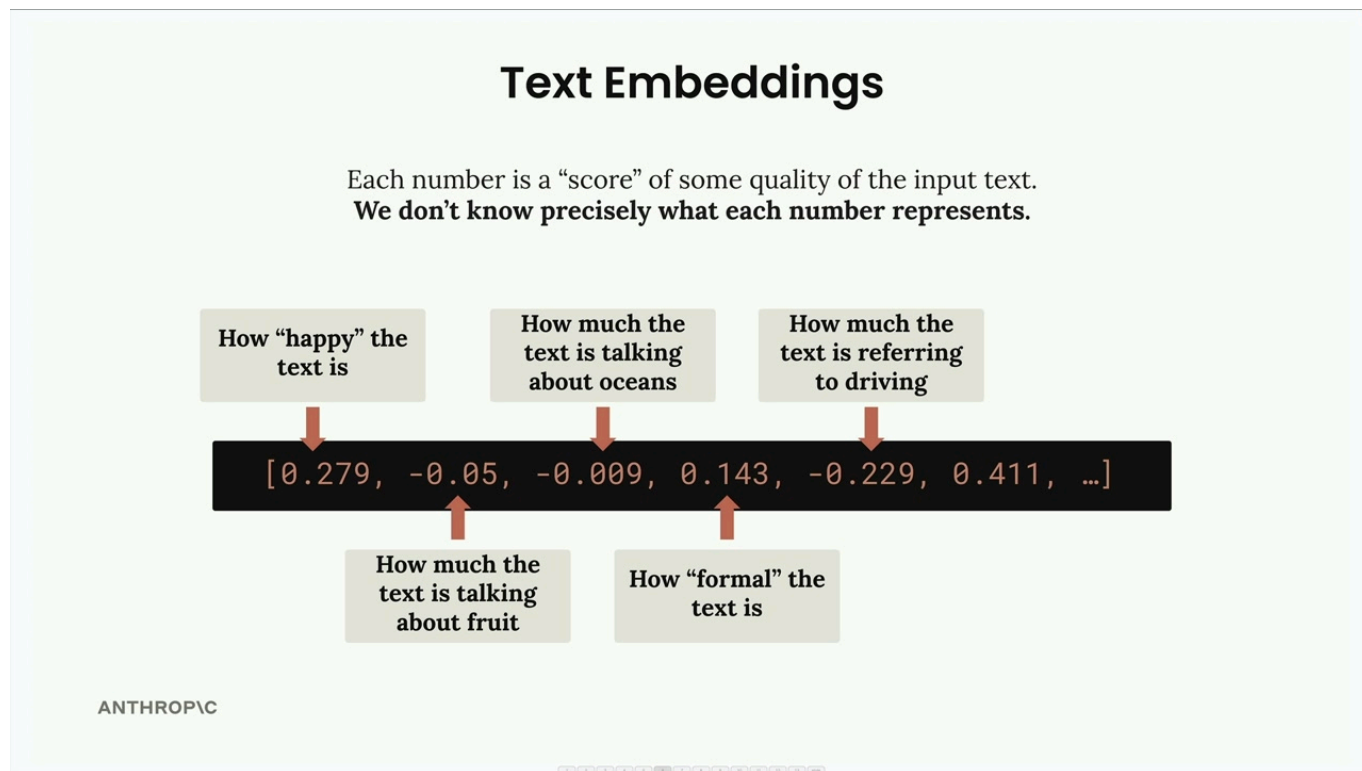
Here's how the process works:

- You feed text into an embedding model
- The model outputs a long list of numbers (the embedding)
- Each number ranges from -1 to +1

- These numbers represent different qualities or features of the input text

Understanding the Numbers

Each number in an embedding is essentially a "score" for some quality of the input text. However, here's the important caveat: we don't know precisely what each number represents.

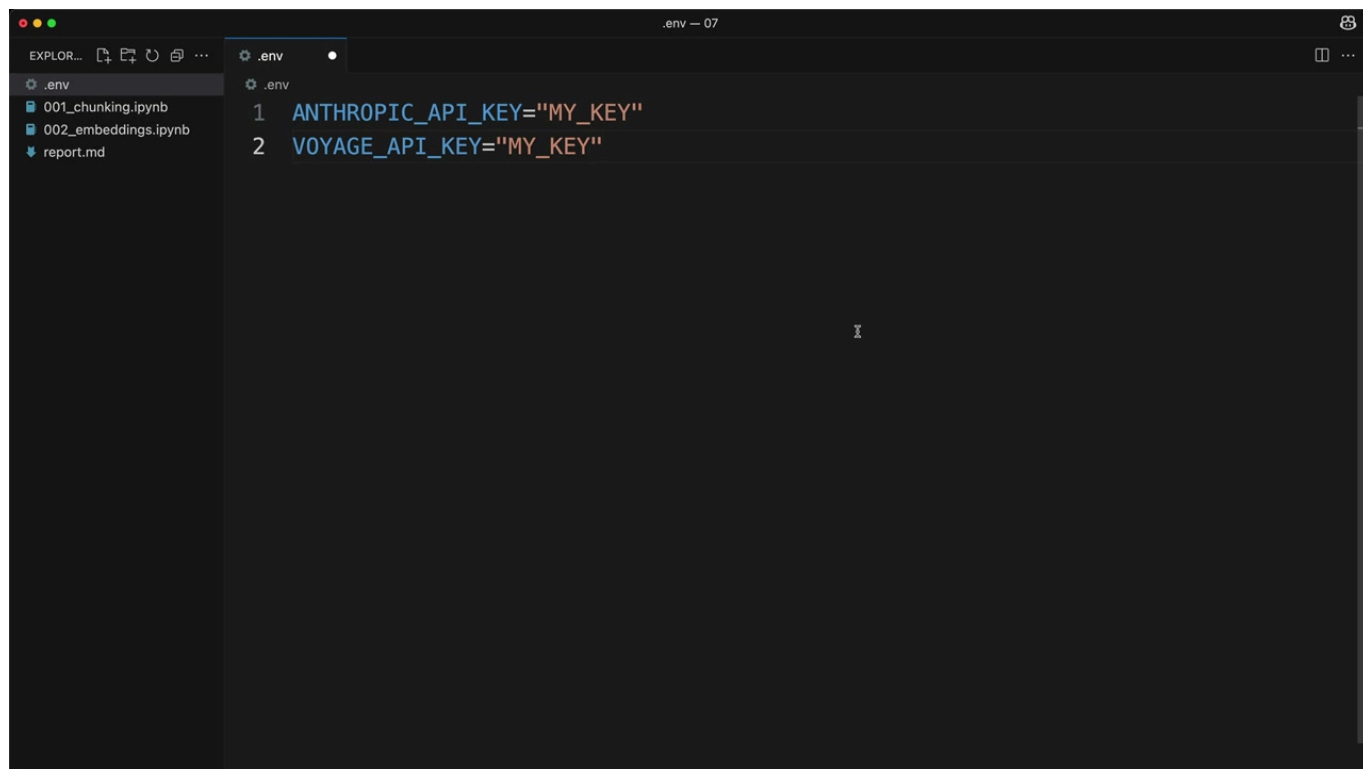


While it's helpful to imagine that one number might represent "how happy the text is" or "how much the text talks about oceans," these are just conceptual examples. The actual meaning of each dimension is learned by the model during training and isn't directly interpretable by humans.

VoyageAI for Embeddings

Since Anthropic doesn't currently provide embedding generation, the recommended provider is VoyageAI. You'll need to:

- Sign up for a separate VoyageAI account
- Get an API key (free to get started)
- Add the key to your environment variables



In your .env file, add:

```
VOYAGE_API_KEY="your_key_here"
```

Implementation

First, install the VoyageAI library:

```
%pip install voyageai
```

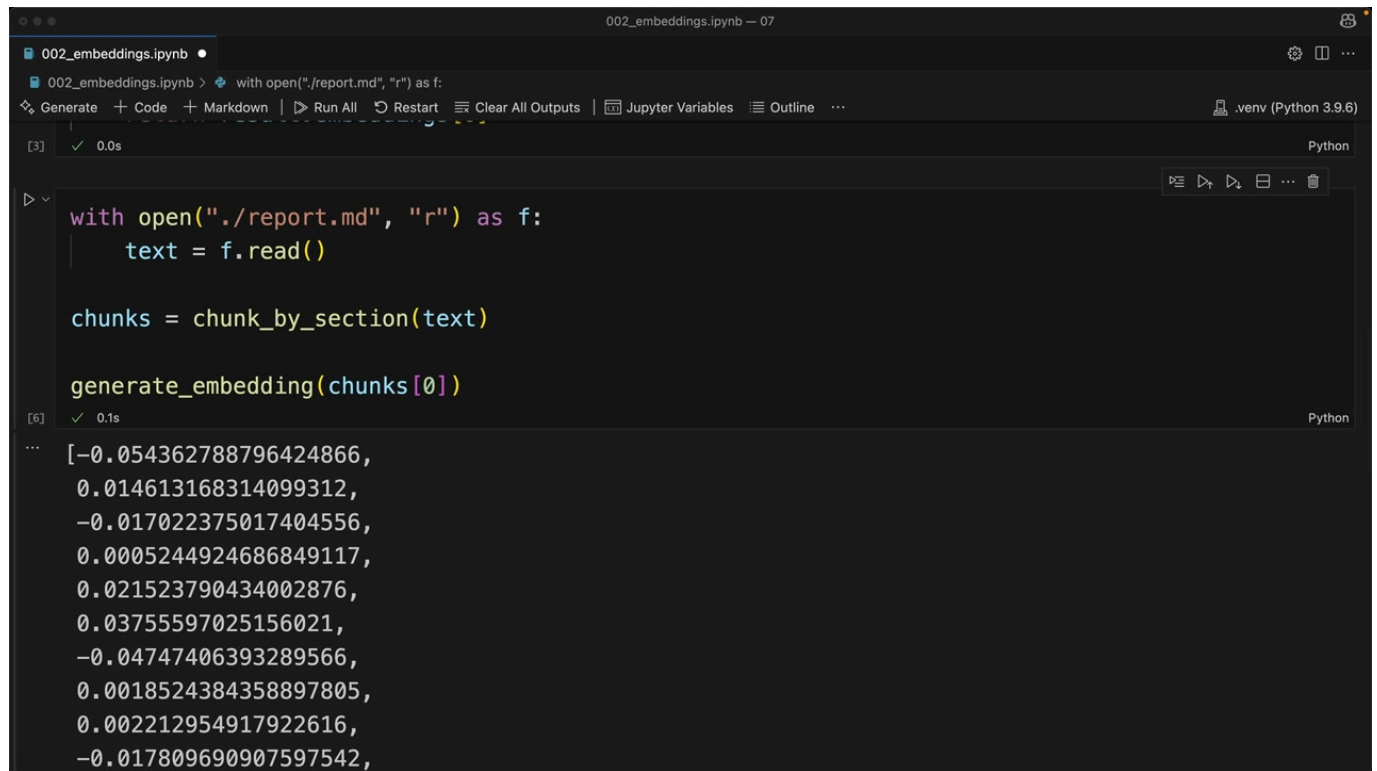
Then set up the client and create a function to generate embeddings:

```
from dotenv import load_dotenv
import voyageai

load_dotenv()
client = voyageai.Client()

def generate_embedding(text, model="voyage-3-large", input_type="query"):
    result = client.embed([text], model=model, input_type=input_type)
    return result.embeddings[0]
```

When you run this function on a text chunk, you'll get back a list of floating-point numbers representing the embedding. The process is quick and straightforward - the real challenge is understanding how to use these embeddings effectively in your RAG pipeline for finding the most relevant content.



A screenshot of a Jupyter Notebook titled '002_embeddings.ipynb'. The notebook is running on a Python 3.9.6 environment. The code in the first cell reads a file named 'report.md' and processes it into chunks. The second cell calls 'generate_embedding' on the first chunk, resulting in a list of 10 floating-point numbers representing the embedding.

```
with open("./report.md", "r") as f:
    text = f.read()

chunks = chunk_by_section(text)

generate_embedding(chunks[0])
```

```
[[-0.054362788796424866,
 0.014613168314099312,
 -0.017022375017404556,
 0.0005244924686849117,
 0.021523790434002876,
 0.03755597025156021,
 -0.04747406393289566,
 0.0018524384358897805,
 0.002212954917922616,
 -0.017809690907597542,
```

The next step is learning how to compare embeddings to determine which chunks are most similar to a user's question, which forms the core of the semantic search process.